

Learning Transfer Functions from Instructions

Amortising visibility search in direct volume rendering
with a measurement grounded in the rendered image

Kevin Kunkel · Master's thesis · Universität Leipzig

19 September 2026

Abstract A transfer function decides what a direct volume rendering shows, and tuning one by hand is the main obstacle to using volume rendering conversationally. We ask whether the mapping from an instruction (“show more bone”, “hide the soft tissue”) and a patient scan to a transfer function can be **learned**, rather than searched for per command. Two things make the question answerable. First, a measurement of what a rendering actually shows, per anatomical class, cheap enough to sit inside a reinforcement-learning loop and validated against label-masked VTK renders (Pearson $r \geq 0.948$ on four of five classes). Second, a formulation in which the policy emits a complete transfer function in a single forward pass, after a ten-step refinement formulation failed. On 200 held-out episodes over six unseen patients, averaged over three training seeds, the policy reaches median attainment $+0.275$ with zero visibility evaluations, against $+0.263$ for a hill-climber allowed ten, and beats every non-search baseline at $p < 0.001$. It is not yet as **reliable** as search: it improves 73 % of instructions against search's 93 %. The deliverable is a working system — a browser viewer that answers spoken or typed instructions, the validated measurement, the trained policy, a preference-collection interface for the human-feedback stage, and a reproducibility apparatus that records which code computed every number.

1. Introduction and motivation

Direct volume rendering turns a CT scan into an image by assigning every voxel intensity a colour and an opacity. That assignment — the **transfer function** (TF) — is what decides whether the ribs are visible, whether skin hides them, and whether the lungs read as empty space or as tissue. It is also, in practice, the part users struggle with: a good TF is found by dragging control points until the picture looks right, and the result does not transfer to the next patient.

The application this thesis targets is conversational: a clinician says “show only the skeleton” or “a bit more lung, less soft tissue”, and the rendering answers. A rule-based command executor can do the obvious thing — move the peak associated with the named tissue — but the obvious thing is frequently wrong, because whether bone becomes **visible** depends on what lies in front of it in that particular patient. Section 5 quantifies this: the rule executor scores below doing nothing.

Search solves it. A coordinate hill-climber over the TF parameters, allowed 200 evaluations, reaches $+0.730$ median attainment — far better than anything else we measured. But each evaluation is a render plus a measurement, and 200 of them is roughly 100 seconds per spoken command. That is unusable interactively, and the problem gets worse, not better, in the setting the thesis ultimately targets: when the objective is a **person's** judgment rather than a formula, search becomes impossible, because a radiologist cannot be asked 200 times which of two renderings they prefer.

This motivates amortisation. If a policy can absorb, across many patients, what “show the ribs” requires, it can answer in one forward pass what search rediscovers from scratch every time. The research question is whether that mapping is learnable at all.

Finding: The contribution is not a new RL algorithm. It is that the question became answerable once the objective was grounded in the rendered image and the formulation stopped asking the policy to learn a search procedure. Both of those were failures first, and both are reported here.

1.1. Why the previous objective made the question unanswerable

An earlier version of this project optimised `mass_fraction`: the area under the opacity curve over bone’s Hounsfield range. It never looked at the scan.

Change to the transfer function	Bone actually visible	mass_fraction
default	0.3 %	0.26
raise the bone peak to maximum	0.3 %	0.43 (“better”)
clear the fat and soft-tissue peaks	4.5 %	0.26 (“no change”)

Table 1: The retired objective rewarded a change that altered nothing on screen and ignored the one that made bone nine times more visible.

Any policy trained against that signal optimises a number disconnected from the image. Everything in this paper follows from replacing it.

2. Problem formulation

A transfer function here is four Gaussian peaks over the Hounsfield range $[-1050, 2000]$, six parameters each (centre, width, height, r, g, b) — 24 values. Peak centres are **fixed** at anatomically motivated positions (lungs -800 HU, soft tissue 40 HU, vessels 300 HU, skeleton 900 HU); the policy controls the remaining 12 groups (height, width and colour per peak).

An **episode** is a triple: a volume V , a start transfer function θ_0 , and an instruction I . The policy must produce θ such that the rendering of V under θ satisfies I . Instructions are sampled from a grammar with five kinds:

- relative** (40 %) — “show moderately more bone”
- compound** (25 %) — “more bone, less soft tissue”
- show_only** (15 %) — “show only the lungs”
- absolute** (10 %) — “make the vessels highly visible”
- brightness** (10 %) — “brighten the skeleton”

Three questions sit underneath, and the pipeline separates them deliberately: **Q1**, can “what the instruction asks for” be measured at all; **Q2**, given such a measure, can a policy satisfy instructions on patients it has never seen; **Q3**, can the measure come from a human instead of a formula. This paper answers Q1 and Q2 and delivers the apparatus for Q3.

3. Method

3.1. Measuring what a rendering shows

Direct volume rendering is too expensive to run inside an RL loop, and a rendered image does not say which tissue produced which pixel. `visibility.py` estimates both cheaply. A volume is resampled once per view into a cube whose first axis runs front to back; that cube is then composited front to back for a given TF. The result is, per class, how much of the final image that class contributes (`vis`), how bright it appears (`bright`), and how much of the frame is covered at all (`coverage`). Six standard views, an 80^3 grid, roughly 15 ms per evaluation on CPU.

Class labels come from TotalSegmentator label volumes. Five material classes are measured — skeleton, lungs, organs, muscle, vessels — and mapped onto four **goal** classes, with organs and muscle combined into `soft`, because no transfer function over intensity separates them.

3.1.1. Validation against real renders

The estimate is only worth as much as its agreement with an actual renderer. The reference renders the same TF twice: once normally, once with one class’s **colour** set to black and its **opacity untouched**, using VTK label-map masking. The difference in mean luminance is exactly that class’s contribution, with occlusion unchanged.

Finding: Deleting a class’s voxels instead — the obvious first method — measures something else, because the holes reveal what lies behind. On `ts_s0454` with a bone-dominant TF: 0.1481 normally, 0.1065 with the skeleton blacked out (its true contribution), 0.1194 with the skeleton **deleted** — higher, because the background showed through. Under the deletion method, thin surface tissue anti-correlated with its own visibility.

Class	ts_s1379	ts_s1337	ts_s0454	Status
skeleton	1.000	0.999	1.000	validated
organs	0.995	0.994	0.969	validated
muscle	0.981	0.948	0.999	validated
vessels	0.992	0.989	(0.001 range)	validated on contrast scans
lungs	(0.0001 range)	(0.0000 range)	absent	below noise floor
coverage	1.000	1.000	1.000	rank agreement

Table 2: Pearson correlation between the estimate (visibility $\times$ brightness) and the label-masked render reference, 10 transfer functions per class. Classes whose reference contribution stays below the noise floor (0.002) are reported unvalidated rather than counted as passes.

Caveat: Lungs cannot be validated this way: lung tissue is nearly transparent, so its luminance contribution sits at the renderer’s noise floor. Lungs remain a usable goal — the estimate tracks lung visibility — but the claim “validated against renders” does not extend to them.

3.2. Instructions, goals and attainment

An instruction is compiled into a **goal vector** of 16 values: per goal class, a target change in $\log_{10}$ visibility d_i , a mask m_i saying whether that class is mentioned, a target brightness change e_i , and its mask n_i . Strengths are fixed in $\log_{10}$ units — **slightly** 0.15, **moderately** 0.3, **strongly** 0.6 — and “show only” hides unnamed classes at strength 1.0, hard enough that a searcher reads it as “push these away” rather than “nudge”.

Distance from a state to the goal is

$$D = \sum_i m_i |c_i - d_i| + \kappa \sum_i n_i |b_i - e_i| + \lambda \sum_i (1 - m_i) \max(0, |c_i| - \tau) + \lambda \kappa \sum_i (1 - n_i) \max(0, |b_i| - \tau)$$

with $\kappa = 1.5$, $\lambda = 0.3$, $\tau = 0.15$, where c_i and b_i are the achieved $\log_{10}$ visibility and brightness changes relative to the start.

Finding: The keep-tolerance τ is load-bearing. Moving one peak inevitably shifts what occludes what elsewhere, so an unmentioned class always drifts a little even when the instruction is followed well. A keep term that punishes any drift at all makes leaving the TF untouched score better than acting on the instruction — the objective would reward inaction.

The reported metric throughout is **attainment**:

$$A = 1 - \frac{D_{\text{final}}}{D_{\text{start}}}$$

so $A = 1$ means the goal was reached, $A = 0$ means no better than doing nothing, and $A < 0$ means worse. Normalising by D_{start} is what makes episodes comparable: start distances range from 0.15 (“brighten the skeleton slightly”) to 3.0 (“show only the lungs”), and without normalisation large-target episodes are worth twenty times more than small ones.

Caveat: Attainment is unbounded below — a single bad episode on an easy goal (small D_{start}) can read -17 and swamp a mean. Every aggregate in this paper is therefore a median, with a mean clipped to $[-1, 1]$ reported beside it. One per-kind table computed as a plain mean once made a kind whose median was $+0.51$ read as -23.5 .

3.3. Policy, observation and action

Component	Definition
Observation (57)	goal vector (16) + volume intensity histogram (16) + per-class $\log_{10}$ visibility (4) + brightness (4) + $\log_{10}$ reachable ceiling (4) + current controllable parameters (12) + coverage (1)
Action (12)	the new value of each controllable group in $[-1, 1]$ — not a delta
Reward	attainment of the resulting TF, clipped to $[-1, 1]$, minus a penalty of 1.0 when the result shows effectively nothing
Episode	one step

Table 3: The one-shot formulation.

The **reachable ceiling** channel deserves a note: `solo_max` is the most of a given anatomy that a single-peak transfer function can show on **this** volume. Without it the policy must infer each scan’s headroom from the intensity histogram alone, and an absolute instruction (“make the vessels highly visible”) has no fixed meaning across patients — its target is a share of what is reachable.

3.3.1. The ten-step formulation, and why it was abandoned

The first formulation gave the policy ten steps of ± 0.1 per parameter, rewarding the drop in goal distance at each step. It did not learn.

Setting	Steps	Median attainment	Verdict
20 volumes, all instruction kinds	20k $\rightarrow$ 120k	$-0.28 \rightarrow -0.13$	flat
1 volume, 1 instruction kind	20k $\rightarrow$ 100k	$-0.15 \rightarrow -0.98$	degrades
+ 4 gradient steps per env step	20k	-1.26	worse
+ lower exploration	20k	-0.42	worse
+ $10\times$ reward scale	20k	-0.08	no better

Table 4: The ten-step formulation does not learn, and gets worse with more training even reduced to a single volume and a single instruction kind.

Finding: The diagnosis is structural, not a hyperparameter. A ten-step policy must commit to each parameter change **blind**, while the hill-climber it is measured against may try a change, measure it, and reject it. The policy was being asked to learn an optimisation procedure — a harder problem than the mapping the thesis asks about, and one where a 10-evaluation search already does well. Emitting the whole transfer function at once removes the search from the policy’s job and leaves the mapping.

4. Experimental setup

4.1. Data

Thirty TotalSegmentator subjects, selected with a fixed seed from 100 candidates meeting a minimum field-of-view (≥ 250 mm in plane, ≥ 150 mm superior-inferior), stratified across four body regions.

Region	Train	Val	Test
whole body	5	1	2
trunk	5	1	2
thorax	5	1	1
abdomen / pelvis	5	1	1
total	20	4	6

Table 5: Subject split. Test subjects ts_s0454, ts_s0477, ts_s0591, ts_s1337, ts_s1379, ts_s1388 are never seen in training or in checkpoint selection.

4.2. Training

SAC (stable-baselines3, MlpPolicy), 150 000 steps per seed, three seeds. Hyperparameters are SB3 defaults with one change: `learning_starts` = 500 rather than 100, giving SAC more random one-step episodes before it learns from a mostly empty replay buffer. Every 10 000 steps the deterministic policy is evaluated on a fixed, seeded set of 40 validation episodes; the checkpoint with the best validation **median** is kept.

Seed	Best validation median	At step	Final (150k)
0	+0.285	100k	+0.250
1	+0.249	140k	+0.246
2	+0.348	140k	+0.333

Table 6: Checkpoint selection. The evaluated checkpoint is the best of 15, not the last — on seed 0 it comes from 100k steps.

Caveat: Selecting the best of 15 checkpoints on 40 episodes is selection on noise: these validation medians are maxima and overstate what a single run would produce. The selection uses only validation subjects, so the held-out numbers in Section 5 remain unbiased, but the validation figures here should not be read as performance estimates.

4.3. Baselines

Baseline	Evaluations	What it does
B0 do nothing	0	returns the start TF; scores exactly 0 by construction, and is the “did acting help” line rather than the floor
B1 rule executor	0	what the current command executor does: move only the peak belonging to each mentioned class
B2 random	0	10 random ± 0.1 steps, ignoring the instruction entirely — the floor any policy must clear
B3 hill-climb (cheap)	10	coordinate search on D
B4 hill-climb (thorough)	200	the same search, $20\times$ the budget
B5 occlusion heuristic	0	B1, plus strip every other peak to height 0.02 — the “hide everything else” rule a person writes by hand

Table 7: Baselines. “Evaluations” counts calls to the visibility measurement, which is the cost that matters interactively.

A hybrid arm, **policy + refinement**, seeds the same coordinate search with the policy’s proposal at a budget of three evaluations, then scores the refined result and keeps whichever of {proposal, refined} is better. Its cost is therefore 4 evaluations: one to score the proposal, two candidate probes inside the search, and one to score the result.

4.4. Episode protocol and statistics

The 200 test episodes are deterministic in the evaluation seed, so **every** arm — policy, hybrid and all six baselines — is scored on identical volumes, start states and instructions. Comparisons are paired Wilcoxon signed-rank tests over episodes (normal approximation with continuity correction, ties dropped and average ranks for tied magnitudes).

Caveat: Two aggregations appear in the literature on this project and they must not be crossed. **Median of seed medians** takes each seed’s median attainment and medians those three numbers. **Seed-averaged per-episode** averages the three seeds’ attainment on each episode first, then takes the median over 200 episodes. The paired p -values belong to the second, because the pairing is per episode. Section 5.3 reports both for the same comparison; they differ by a factor of three.

5. Results

5.1. Held-out performance

200 instructions, six unseen patients, three seeds, every arm on identical episodes.

Method	Evals	Median	Clipped mean	Improved	vs. policy
B4 hill-climb (thorough)	200	+0.730	+0.643	100 %	better, $p < 0.001$
policy + 3 refinements	4	+0.316	+0.254	76 %	—
policy alone	0	+0.275	+0.205	73 %	—
B3 hill-climb (cheap)	10	+0.263	+0.318	93 %	split by seed
B0 do nothing	0	0.000	0.000	—	worse, $p < 0.001$
B1 rule executor	0	− 0.022	− 0.172	37 %	worse, $p < 0.001$
B2 random	0	− 0.040	− 0.078	39 %	worse, $p < 0.001$
B5 occlusion heuristic	0	− 0.191	− 0.290	30 %	worse, $p < 0.001$

Table 8: Held-out attainment. Median and clipped mean are medians over the three seeds; “improved” is their mean. Baseline rows do not vary by seed — the episode seed is fixed, so only the policy differs between the three runs.

Finding: The policy reaches the 10-evaluation hill-climber’s median while spending **no** evaluations at all (+0.275 vs +0.263), and the hybrid exceeds it at 4 evaluations (+0.316). Against every non-search baseline the policy wins at $p < 0.001$. Note that B1, the rule-based executor this system would otherwise ship, scores **below doing nothing**: acting on the named tissue alone is worse than leaving the transfer function untouched.

5.1.1. Why the rule executor fails, concretely

B1’s median hides a bimodal split, and the mechanism is worth stating because it is the thesis’s argument in miniature. Swept over 53 instruction-volume pairs in the viewer, the rule executor’s median attainment is +0.003 — it improves 28 and **worsens** 25. But by instruction kind it ranges from +0.147 on show-only to −2.192 on compound.

The show-only case is the instructive one. `commands.apply_command` answers “show only X” by setting X’s peak to a hard-coded height of 0.7 and every other peak to 0, while the goal asks for X to become ten times more visible (`HIDE_STRENGTH = 1.0` in $\log_{10}$). Default peak heights are lungs 0.05, soft tissue 0.15, vessels 0.30, skeleton 0.60. So:

“**Show only the lungs**” moves the lung peak $0.05 \rightarrow 0.7$ — a $14\times$ jump against a goal asking for $10\times$. The rule succeeds, and beats even the 200-evaluation search on four of five test subjects.

“**Show only the skeleton**” moves the bone peak $0.60 \rightarrow 0.7$ — a $1.17\times$ nudge against the same $10\times$ goal, while dimming everything else. It lands **below doing nothing** on four of five subjects.

Finding: The rule is not bad at “show only”. It is good at showing things that started dim, because a constant cannot know what the current state is. That is the whole case for grounding the decision in a measurement of the render: the quantity a rule would need to know — how far this tissue is from where the instruction wants it, on **this** scan — is exactly what the rule has no access to, and what both search and the policy consume.

Caveat: These sweep numbers come from single episodes at the default transfer function, not from the held-out protocol of Section 5, and are reported to explain a mechanism rather than to measure one. The held-out figure for B1 remains −0.022 over 200 episodes.

Caveat: The comparison against cheap search splits by seed: seed 0 favours search ($p = 3.0 \times 10^{-4}$), seeds 1 and 2 favour the policy ($p = 0.054, p = 0.029$). The defensible claim is that the policy **matches** cheap search at a fraction of its cost, not that it beats it.

Caveat: Reliability is the honest headline and the gap is not small. Search improves 93 % of episodes, the policy 73 %, the hybrid 76 %; search’s clipped mean (+0.318) is also the better figure. The policy matches cheap search **in the middle of the distribution** while being cheaper. Its worst episodes are worse.

5.2. Where the policy succeeds and fails

Instruction kind	Median (seed 1)	Episodes	Share of mix
absolute	+0.49	27	10 %
brightness	+0.48	30	10 %
relative	+0.30	74	40 %
show only	+0.20	26	15 %
compound	+0.09	43	25 %

Table 9: Per instruction kind. Compound instructions — two constraints at once — are a quarter of the sampled mix and 43 of the 200 test episodes, and the policy essentially fails them.

Finding: Compound instructions are the clearest capability gap, and the one a user notices first: “more bone, less soft tissue” is an ordinary thing to say. The one-shot formulation asks the policy to satisfy two constraints in a single output, with no opportunity to trade them off against a measurement.

5.3. Does the corrected observation help?

The reachable-ceiling channel was initially probed at the wrong peak, and a colour action collapsed r , g , b to one scalar. Three seeds were retrained after both were fixed (v3) and compared against three trained before (v2), on the same 200 episodes.

Statistic	v2	v3	Gap
median of seed medians	+0.247	+0.275	+0.028
median of seed-averaged per-episode	+0.201	+0.286	+0.086

Table 10: The same comparison under both aggregations. Paired Wilcoxon over the seed-averaged per-episode values: $p = 0.0064$, v3 ahead on 59 % of episodes.

Kind	v2	v3	Delta	n
absolute	+0.366	+0.489	+0.122	27
brightness	+0.345	+0.411	+0.066	30
relative	+0.185	+0.239	+0.054	74
compound	+0.017	+0.055	+0.038	43
show only	+0.164	+0.181	+0.017	26

Table 11: Where the gain lands, seed-averaged per episode.

Finding: The gain concentrates in **absolute** instructions, which are exactly the ones whose targets are a share of the reachable ceiling — the channel that was repaired. That mechanism-consistent concentration, rather than the overall gap, is the evidence that the ceiling channel earns its place in the observation.

Caveat: The effect is real but fragile, and this paper does not claim more than that. It triples depending on a defensible choice of aggregation (+0.028 vs +0.086); $n = 3$ training runs per arm; and the p -value is a paired test over 200 **episodes**, so it speaks to consistency across instructions rather than across

seeds. An earlier ablation that would have isolated the channel directly (“policy, no ceiling input”) was never validly run — the stored result file is byte-identical to its own control — and is withdrawn rather than repaired.

5.4. Convergence

Caveat: Earlier drafts of this work stated that the runs had not converged and that training longer was therefore the cheapest improvement. The validation curves do not support that. Relative and compound instructions start deeply negative (-0.85 , -0.50) and climb steeply, but over the final five evaluations they are flat or falling — relative $0.15/0.10/0.14/0.18/0.16$ on seed 0, compound $0.12 \rightarrow 0.04$ on seed 0 and $0.26 \rightarrow 0.14$ on seed 2 — and seed 0’s overall validation median peaks at 100k steps and ends lower. The kinds the policy handles badly **plateau** well below the kinds it handles well. That reads as a capability gap in the one-shot formulation rather than an exhausted step budget. Training longer remains worth trying; it is no longer the conclusion the figure demonstrates.

6. Methodology: how these numbers are kept honest

This section is included deliberately, because the project’s most instructive result is a measurement failure it caught itself.

6.1. The incident

A batch evaluation started in the evening, a scoring fix landed in `goals.py` and `visibility.py` at 22:48 while it ran, and the job wrote its result files at 03:26 — carrying numbers computed by the code it had **imported** six hours earlier. The files’ modification times were newer than the fix, so nothing about them looked wrong. A week of thesis numbers was published from them, including two conclusions that did not survive correction: the policy’s attainment ($+0.169$, actually $+0.275$) and a null result on retraining ($p = 0.85$, actually $p = 0.0064$ in the opposite direction, Section 5.3).

Finding: Every baseline moved up together with the policy under re-measurement. That is what identifies the fault as the **ruler** rather than the method — a policy that had genuinely improved would not have dragged a hand-written heuristic up with it.

6.2. What changed so it cannot recur

Every result file now records, captured **at import time** rather than at write time, the git commit, whether the tree was dirty, and a fingerprint of the scoring modules actually loaded. Re-printing a stored result checks that fingerprint against the code on disk and prints a stale-result banner when they diverge. A long-running job must report the code it loaded, not what git happens to say when it finally writes.

Result files additionally store per-episode rows — attainment, instruction kind and volume for every episode and every arm. An aggregate cannot be re-aggregated: the comparison in Section 5.3 needs the median of **seed-averaged per-episode** attainment, which is not recoverable from a stored median, and for a period it was quoted from a session that no longer existed. Storing the rows means a later question is answered from the files rather than from another evaluation run.

Apparatus	Purpose
<code>provenance.py</code>	commit, tree state and scoring-module fingerprint in every result file
<code>rl.vis_eval --show</code>	re-prints a stored result and flags it stale if the scoring code has moved
<code>episodes_detail</code>	per-episode rows, so any later aggregation is computed from files
<code>tools.compare_runs</code>	paired comparison of two groups of runs, from stored rows, never re-running a policy
<code>docs/REPRODUCE.md</code>	every command from raw data to every number and figure
611 tests	including a hand-computed Wilcoxon example and the measurement’s own validation

Table 12: The reproducibility apparatus.

Finding: The fingerprint covers whole modules rather than scoring lines, so an edit that cannot change a number still invalidates older files. This is deliberately blunt in that direction: it would rather send an author back to a re-run than let a changed module pass unnoticed. When a re-run reproduces the old numbers exactly — as all six did here, to six decimal places — that is the evidence the edit was cosmetic.

7. Deliverable

The thesis deliverable is a working system, not a set of numbers.

Component	State
Conversational viewer (/)	Browser UI over a local server. Speak or type an instruction; a local LLM parser (with a rule-grammar fallback when the LLM is unreachable) turns it into a command; the transfer function is rewritten and the render updates.
Three answering modes	exact (apply the parsed command directly, the default), search (hill-climb), policy (the trained checkpoint).
Visibility measurement	<code>visibility.py</code> , validated against label-masked VTK renders on four of five classes.
Trained policy	Three seeds on the corrected pipeline; the viewer loads seed 0.
Preference collector (/collect)	Two candidate renderings and an instruction; the rater picks which better answers it. Pairs too similar to distinguish are discarded, one item in ten repeats an earlier one with sides swapped to measure rater self-consistency.
Reproducibility apparatus	As tabulated above.

Table 13: What the thesis delivers.

Caveat: The viewer’s default answering mode is **exact**, not **policy**, and this is an honest statement of where the policy currently stands: on “show only bones” it leaves soft tissue dominant (43.1 %) with the skeleton at 4.9 %, where direct application isolates it (skeleton 16.7 %, lungs 0.0 %). The policy proposes a direction; it does not execute an instruction. It is one click away in the toolbar.

Caveat: The viewer loads the seed-0 checkpoint, which is the **weakest** of the three on held-out data (+0.231, against +0.307 and +0.275). The choice was fixed before the held-out numbers were known and has not been revisited; a demonstration using seed 1 would look better and would need to say so.

7.1. Human preferences: built, not yet populated

The apparatus for Q3 — learning the objective from a person rather than a formula — is complete and running, and has **no clean data behind it**. Fifty-four judgments were collected before a checkpoint-selection defect was found (the collector was sampling the superseded v2 policy while the viewer used v3) and before a colour fix that made candidate pairs visually blind; they are quarantined as a pilot. The clean set stands at zero.

Caveat: This is the largest gap between what the architecture describes and what has been measured. Nothing in the results above depends on preference data — the objective throughout this paper is the validated formula, not a learned reward model — but the conversational-objective argument in Section 1 remains an argument, not a result.

8. Limitations

- Lungs are a goal but not validated against renders; their luminance contribution sits below the renderer’s noise floor.
- Organs and muscle are one target, because no intensity-based transfer function separates them.
- Vessels are a goal only on contrast scans (5 of 30 volumes), and the per-class vessel statistics rest on 17 episodes.
- Peak positions are fixed; only height, width and colour are learned. Lung goals are reachable only because a peak is seeded at -800 HU.
- Generalisation is claimed from 20 training and 6 test subjects.
- The policy matches cheap search on the median but not on reliability (73 % vs 93 % of episodes improved); its worst episodes are worse.
- Compound instructions — a quarter of the mix — are close to not working.
- Three seeds per arm. With this seed-to-seed spread, only a large effect would be detectable across runs.
- The per-class analysis of which anatomy benefits most from retraining was measured in the stale batch and has not been re-run; per-episode rows currently record instruction kind and volume, not goal class.

9. Conclusion

The question this thesis asks — can a transfer function be **learned** from an instruction — is answered affirmatively, with two qualifications that matter. A policy trained on 20 patients produces, in a single forward pass with no visibility evaluations, transfer functions that match on the median what a coordinate search reaches with ten, on six patients it has never seen, and that beat every non-search baseline including the rule-based executor such a system would otherwise ship. It does so less **reliably** than search, and it largely fails the compound instructions that make up a quarter of ordinary phrasing.

Both qualifications point the same way: the one-shot formulation removed the search from the policy’s job, which is what made learning work, and the capability that remains missing — trading two constraints off against each other — is the part of search that was removed. A formulation with a small measured budget, of which the 4-evaluation hybrid arm is the crudest possible version, is where the next result probably lies.

All numbers in this document come from result files in `out/rl_v2/` carrying a provenance record, and the commands that reproduce each of them are in `docs/REPRODUCE.md`. The held-out table and Section 5.3 were re-derived independently on 19 September 2026 and reproduced the stored values exactly.